

Panduan Rewrite: Express + TypeScript Decorator Router

[Catatan Penting Sebelum Mulai](#)

[1. Ringkasan Proyek](#)

[2. Prasyarat & Yang Perlu Diinstal](#)

[2.1 Software dasar](#)

[2.2 Dependency dari package.json Anda](#)

[2.3 Yang hilang dari package.json Anda — perlu ditambahkan](#)

[3. Langkah demi Langkah: Membangun Ulang Proyek dari Nol](#)

[Langkah 0 — Inisialisasi proyek](#)

[Langkah 1 — tsconfig.json](#)

[Langkah 2 — Enum dasar](#)

[Langkah 3 — AppRouter.ts \(Singleton\)](#)

[Langkah 4 — routes.ts \(decorator method HTTP\)](#)

[Langkah 5 — use.ts \(decorator middleware per-route\)](#)

[Langkah 6 — bodyValidator.ts \(decorator validasi body\)](#)

[Langkah 7 — controller.ts \(class decorator utama\)](#)

[Langkah 8 — decorators/index.ts \(barrel export\)](#)

[Langkah 9 — Controller sungguhan](#)

[Langkah 10 — loginRoutes.ts — perlu konfirmasi Anda](#)

[4. Entry Point \(src/index.ts \) — Rekonstruksi, Bukan File Asli](#)

[5. Alur Request Secara Keseluruhan \(Ringkasan\)](#)

[6. Bug yang Ditemukan pada Kode Asli & Kenapa Itu Bug](#)

[6.1 controller.ts — tipe keys salah \(nyata, terverifikasi di file asli\)](#)

[6.2 controller.ts — hanya cek path, tidak cek method](#)

[6.3 controller.ts — tidak melewati constructor secara eksplisit](#)

[6.4 package.json — typescript dan @types/node tidak terdaftar](#)

[7. Cara Menjalankan Proyek](#)

[8. Rekomendasi Lanjutan](#)

Catatan Penting Sebelum Mulai

Dokumen ini disusun berdasarkan **file asli yang Anda unggah**: `AppRouter.ts`, `controller.ts`, `routes.ts`, `use.ts`, `bodyValidator.ts`, `Methods.ts`, `MetadataKeys.ts`, `decorators/index.ts`, `LoginController.ts`, `RootController.ts`, `loginRoutes.ts`, `tsconfig.json`, `package.json`, `package-lock.json`.

Ada **dua hal yang tidak bisa saya verifikasi** dan saya tandai secara eksplisit di bagian terkait supaya Anda tidak salah asumsi:

- src/index.ts (entry point aplikasi) tidak ada di antara file yang diunggah.** Dua file yang Anda beri nama `index.ts` ternyata isinya sama — keduanya adalah `decorators/index.ts` (barrel export), bukan file yang menjalankan `express()`, memasang middleware, dan `app.listen(...)`. **[Confidence: Low untuk bagian entry point]** — bagian tersebut di bawah adalah rekonstruksi berdasarkan dependency di `package.json` dan pemakaian `req.session` di controller, **bukan kode asli Anda**. Mohon bandingkan dengan file asli Anda.
- loginRoutes.ts tampaknya kode versi lama (non-decorator)** yang fungsinya tumpang tindih dengan `LoginController.ts` (versi decorator). Tanpa melihat entry point asli, saya tidak bisa memastikan apakah file ini masih dipakai (di-mount manual) atau sudah tidak terpakai. Ditandai di bagian relevan.

Sisanya (semua file decorator, controller, konfigurasi) sudah saya baca langsung dari isi file Anda — bukan tebakan.

1. Ringkasan Proyek

Proyek ini adalah server **Express** yang ditulis dengan **TypeScript**, memakai pola *decorator* (mirip NestJS) untuk mendefinisikan route secara deklaratif menggunakan `@controller`, `@get`, `@post`, `@use`, `@bodyValidator`. Di baliknya, semua decorator menyimpan data (path, method, middleware, validator) sebagai *metadata* pada method class memakai library `reflect-metadata`, lalu `@controller` membaca metadata itu untuk mendaftarkan route sungguhan ke Express `Router`.

Struktur folder (`src/`):

```
src/
├── routes/
│   └── loginRoutes.ts      <- versi lama, non-decorator (perlu konfirmasi masih dipakai/tidak)
├── controllers/
│   ├── LoginController.ts  <- route auth (login/logout), versi decorator
│   ├── RootController.ts   <- route utama & protected, versi decorator
│   └── decorators/
│       ├── controller.ts   <- @controller (class decorator, "otak" dari sistem ini)
│       ├── routes.ts       <- @get, @post, @put, @del, @patch
│       ├── use.ts          <- @use (menambah middleware per-route)
│       ├── bodyValidator.ts <- @bodyValidator (validasi field body wajib)
│       ├── Methods.ts      <- enum nama HTTP method
│       ├── MetadataKeys.ts <- enum nama key metadata
│       └── index.ts        <- barrel export semua decorator
├── index.ts                <- entry point (TIDAK diunggah, lihat Bagian 8)
└── AppRouter.ts            <- singleton Express Router
```

2. Prasyarat & Yang Perlu Diinstal

2.1 Software dasar

- **Node.js** (disarankan versi LTS terbaru) — sudah termasuk `npm`.
- Editor kode (VS Code, dsb).

2.2 Dependency dari `package.json` Anda

Package	Kategori	Untuk apa
<code>express</code>	runtime	Web framework utama — menerima request HTTP dan mengarahkan ke handler
<code>reflect-metadata</code>	runtime	Polyfill Reflection API — dipakai semua decorator untuk <code>Reflect.defineMetadata</code> / <code>getMetadata</code>
<code>cookie-session</code>	runtime	Menyimpan session login user di cookie (dipakai <code>req.session</code> di controller)
<code>body-parser</code>	runtime	Mem-parsing body request (form/JSON) menjadi <code>req.body</code>
<code>nodemon</code>	runtime (dev tool)	Auto-restart server saat file <code>.js</code> hasil build berubah
<code>concurrently</code>	runtime (dev tool)	Menjalankan <code>tsc -w</code> dan <code>nodemon</code> bersamaan lewat satu perintah <code>npm start</code>
<code>@types/express</code> , <code>@types/body-parser</code> , <code>@types/cookie-session</code>	types	Supaya TypeScript mengenali tipe dari library JS di atas

2.3 Yang hilang dari `package.json` Anda — perlu ditambahkan

Saya cek `package-lock.json` Anda dan `typescript` **tidak ada sama sekali** di dependency tree. Script `"start:build": "tsc -w"` akan gagal (`tsc: command not found`) kecuali Anda punya TypeScript global di komputer. `@types/node` juga tidak ada, padahal dipakai banyak API Node bawaan.

[High confidence] — ini berdasarkan pencarian langsung string `"typescript"` di `package-lock.json` Anda, hasilnya nihil.

Perbaikan — tambahkan sebagai devDependency:

```
npm install --save-dev typescript @types/node
```

3. Langkah demi Langkah: Membangun Ulang Proyek dari Nol

Urutan di bawah mengikuti urutan *dependency* antar file — file yang tidak bergantung pada file lain dibuat lebih dulu.

Langkah 0 — Inisialisasi proyek

```
mkdir server && cd server
npm init -y
npm install express reflect-metadata cookie-session body-parser
npm install --save-dev typescript @types/node @types/express @types/body-parser @types/cookie-session nodemon concurrently
```

Langkah 1 — `tsconfig.json`

Buat dengan `npx tsc --init`, lalu pastikan opsi berikut aktif (ini yang ada di `tsconfig.json` Anda):

```
{
  "compilerOptions": {
    "target": "es2016",
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true,
    "module": "commonjs",
    "rootDir": "./src",
    "outDir": "./build",
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "strict": true,
    "skipLibCheck": true
  }
}
```

Mengapa opsi ini penting: - `experimentalDecorators: true` — **wajib**, karena seluruh proyek ini bergantung pada sintaks `@controller`, `@get`, `dst`. Tanpa ini, TypeScript akan menolak compile. - `emitDecoratorMetadata: true` — membuat TypeScript menyisipkan info tipe parameter/return ke metadata `reflect-metadata`. Meski di proyek ini metadata custom (path/method/dll) yang dipakai bukan metadata tipe otomatis, opsi ini tetap diperlukan agar ekosistem decorator + `reflect-metadata` berjalan konsisten. - `rootDir/outDir` — source di `src/`, hasil compile ke `build/`, sesuai dengan script `nodemon build/index.js`. - `strict: true` — inilah yang **akan menangkap bug tipe** yang dijelaskan di Bagian 7 (`keys: string` vs `string[]`) sebagai error compile — pastikan Anda benar-benar menjalankan `tsc` dan membaca errornya, bukan hanya `nodemon` di atas build lama.

Langkah 2 — Enum dasar

`src/controllers/decorators/MetadataKeys.ts`

```
export enum MetadataKeys {
  method = 'method',
  path = 'path',
  middleware = 'middleware',
```

```
    validator = 'validator'
  }
```

Kenapa pakai enum, bukan string biasa? Supaya nama key metadata ('path', 'method', dst.) tidak ditulis berulang sebagai string literal di banyak file — kalau salah ketik, TypeScript akan menandainya sebagai error, bukan bug yang baru diketahui saat runtime.

`src/controllers/decorators/Methods.ts`

```
export enum Methods {
  get = 'get',
  post = 'post',
  patch = 'patch',
  del = 'delete',
  put = 'put'
}
```

Kenapa `del = 'delete'` ? Karena `delete` adalah reserved word di JavaScript/TypeScript, sehingga nama key enum tidak boleh persis `delete`; nilainya ('delete') tetap sesuai nama method Express yang sebenarnya (`router.delete(...)`).

Langkah 3 — `AppRouter.ts` (Singleton)

```
import express from 'express';

export class AppRouter {
  private static instance: express.Router;

  static getInstance(): express.Router {
    if (!AppRouter.instance) {
      AppRouter.instance = express.Router();
    }
    return AppRouter.instance;
  }
}
```

Apa fungsinya & cara membacanya: Ini pola *Singleton* — memastikan hanya ada **satu** instance `express.Router()` di seluruh aplikasi, walau `AppRouter.getInstance()` dipanggil dari banyak file controller berbeda. Setiap kali `@controller` di file manapun mendaftarkan route baru, semuanya menempel ke Router yang sama, yang nantinya dipasang sekali ke `app` di entry point.

Langkah 4 — `routes.ts` (decorator method HTTP)

```
import 'reflect-metadata';
import { RequestHandler } from 'express';
import { Methods } from './Methods';
import { MetadataKeys } from './MetadataKeys';

interface RouteHandlerDescriptor extends PropertyDescriptor {
  value?: RequestHandler;
}

function routeBinder(method: string) {
  return function (path: string) {
    return function (target: any, key: string, desc: RouteHandlerDescriptor) {
      Reflect.defineMetadata(MetadataKeys.path, path, target, key);
      Reflect.defineMetadata(MetadataKeys.method, method, target, key);
    };
  };
}

export const get = routeBinder(Methods.get);
export const put = routeBinder(Methods.put);
export const post = routeBinder(Methods.post);
export const del = routeBinder(Methods.del);
export const patch = routeBinder(Methods.patch);
```

Apa fungsinya: `routeBinder` adalah *decorator factory* tiga lapis — `routeBinder(method)` mengembalikan fungsi yang menerima `path`, yang mengembalikan decorator sungguhan. Saat Anda menulis `@get('/login')` di atas sebuah method, decorator ini **tidak** langsung mendaftarkan route ke Express. Ia hanya **menempelkan**

metadata (`path` dan `method`) ke method tersebut lewat `Reflect.defineMetadata` . Pendaftaran route sungguhan baru terjadi nanti di `controller.ts` .

Cara membaca alurnya: `@get('/login')` → panggil `routeBinder('get')` → hasilnya fungsi `(path) => ...` → dipanggil dengan `'/login'` → hasilnya decorator method yang dijalankan TypeScript otomatis saat class di-compile, dengan `target` = prototype class, `key` = nama method (`getLogin`).

Langkah 5 — `use.ts` (decorator middleware per-route)

```
import 'reflect-metadata';
import { RequestHandler } from 'express';
import { MetadataKeys } from './MetadataKeys';

export function use(middleware: RequestHandler) {
  return function (target: any, key: string, desc: PropertyDescriptor) {
    const middlewares = Reflect.getMetadata(MetadataKeys.middleware, target, key) || [];

    Reflect.defineMetadata(
      MetadataKeys.middleware,
      [...middlewares, middleware],
      target,
      key
    );
  };
}
```

Apa fungsinya: Memungkinkan Anda menambahkan middleware Express (seperti fungsi `requireAuth` di `RootController.ts`) langsung di atas method route, contoh:

```
@get('/protected')
@use(requireAuth)
getProtected(req: Request, res: Response) { ... }
```

Setiap kali `@use(...)` dipanggil di method yang sama (misalnya ditumpuk beberapa kali), ia **membaca middleware yang sudah ada** dari metadata (`[]` untuk kasus belum ada), lalu menambahkan middleware baru ke array itu dan menyimpannya kembali — jadi mendukung banyak middleware per route.

Langkah 6 — `bodyValidator.ts` (decorator validasi body)

```
import 'reflect-metadata';
import { MetadataKeys } from './MetadataKeys';

export function bodyValidator(...keys: string[]) {
  return function (target: any, key: string, desc: PropertyDescriptor) {
    Reflect.defineMetadata(MetadataKeys.validator, keys, target, key);
  };
}
```

Apa fungsinya: Menyimpan daftar nama field yang **wajib ada** di `req.body` untuk route tertentu, misalnya `@bodyValidator('email', 'password')` di `postLogin` . Field ini nanti dibaca oleh `bodyValidators` (perhatikan: nama beda — jamak — didefinisikan di `controller.ts`) untuk membuat middleware validasi otomatis.

Catatan koreksi dari analisis PDF sebelumnya: Pada dokumen sebelumnya saya sempat menduga file ini dan `use.ts` kekurangan `import` , karena saat itu Anda hanya mengirim cuplikan kode (stub). Setelah melihat file **asli** Anda sekarang, dugaan itu **salah** — kedua file sudah mengimpor `reflect-metadata` dan `MetadataKeys` dengan benar sejak awal. Saya koreksi di sini secara terbuka.

Langkah 7 — `controller.ts` (class decorator utama)

Ini bagian paling kompleks — decorator ini **membaca semua metadata** yang ditempel oleh Langkah 4-6, lalu benar-benar mendaftarkan route ke `AppRouter` .

Kode asli Anda:

```
function bodyValidators(keys: string): RequestHandler {
  return function(req: Request, res: Response, next: NextFunction) {
    if (!req.body) {
      res.status(422).send('Invalid request');
      return;
    }
    for (let key of keys) {
      if (!req.body[key]) {
        res.status(422).send(`Missing property ${key}`);
        return;
      }
    }
    next();
  };
}
```

```
if (path) {
  router[method](`${routePrefix}${path}`, ...middlewares, validator, routeHandler);
}
```

Bug nyata yang ditemukan (lihat Bagian 7 untuk detail & alasan): 1. `keys: string` seharusnya `keys: string[]`. 2. `if (path)` sebaiknya `if (path && method)`, dan sebaiknya melewati `constructor` secara eksplisit.

Versi yang sudah diperbaiki (dipakai di Bagian 6 sebagai kode final):

```
import 'reflect-metadata';
import { Request, Response, RequestHandler, NextFunction } from 'express';
import { AppRouter } from '../AppRouter';
import { Methods } from './Methods';
import { MetadataKeys } from './MetadataKeys';

function bodyValidators(keys: string[]): RequestHandler {
  return function(req: Request, res: Response, next: NextFunction) {
    if (!req.body) {
      res.status(422).send('Invalid request');
      return;
    }

    for (let key of keys) {
      if (!req.body[key]) {
        res.status(422).send(`Missing property ${key}`);
        return;
      }
    }

    next();
  };
}

export function controller(routePrefix: string) {
  return function(target: Function) {
    const router = AppRouter.getInstance();

    Object.getOwnPropertyNames(target.prototype).forEach((key) => {
      if (key === 'constructor') return;

      const routeHandler = target.prototype[key];
      const path = Reflect.getMetadata(MetadataKeys.path, target.prototype, key);
      const method: Methods = Reflect.getMetadata(MetadataKeys.method, target.prototype, key);

      const middlewares =
        Reflect.getMetadata(MetadataKeys.middleware, target.prototype, key) || [];
      const requiredBodyProps =
        Reflect.getMetadata(MetadataKeys.validator, target.prototype, key) || [];
      const validator = bodyValidators(requiredBodyProps);

      if (path && method) {
        router[method](
          `${routePrefix}${path}`,
          ...middlewares,
          validator,
          routeHandler
        );
      }
    });
  };
}
```

Cara membaca alurnya, langkah demi langkah saat class seperti `LoginController` di-load: 1.

`Object.getOwnPropertyNames(target.prototype)` mengambil semua nama method di class (`getLogin`, `postLogin`, `getLogout`, ...). 2. Untuk tiap method, baca kembali metadata yang ditempel `@get` / `@post` (path, method), `@use` (middleware), `@bodyValidator` (validator keys) — inilah kenapa urutan Langkah 4–6 harus dibuat **sebelum** `controller.ts`, karena `controller.ts` bergantung pada `MetadataKeys` yang sama. 3. Bungkus `requiredBodyProps` jadi middleware `validator` lewat `bodyValidators(...)`. 4. Kalau method itu memang route (punya `path` dan `method` — dua-duanya harus valid), daftarkan ke Express Router sungguhan: `router[method](path, ...middlewares, validator, routeHandler)`.

Langkah 8 — `decorators/index.ts` (barrel export)

```
export * from './controller';
export * from './routes';
export * from './use';
export * from './bodyValidator';
```

Fungsinya: Supaya file controller (`LoginController.ts`, `RootController.ts`) bisa `import { get, controller, use, bodyValidator, post } from './decorators'` dalam satu baris, tanpa perlu tahu file spesifik di dalam folder `decorators/`.

Langkah 9 — Controller sungguhan

`RootController.ts` dan `LoginController.ts` — ini adalah kode aplikasi Anda yang memakai semua decorator di atas. Isinya sudah benar sesuai yang Anda unggah, tidak ada perubahan.

Poin penting cara membacanya: - `@controller('/auth')` di atas `class LoginController` → semua route di dalamnya otomatis diberi prefix `/auth` (lihat `routePrefix` di Langkah 7). - `@controller('')` di `RootController` → tanpa prefix, jadi route `/` dan `/protected` langsung di root. - Urutan decorator method **dari atas ke bawah dieksekusi dari bawah ke atas** (perilaku standar decorator TS): pada typescript `@get('/protected')` `@use(requireAuth)` `getProtected(...)` `@use(requireAuth)` dijalankan lebih dulu (menaruh metadata middleware), baru `@get('/protected')` (menaruh metadata path/method) — tapi karena keduanya cuma menyimpan metadata terpisah di key yang berbeda, urutan ini tidak mengubah hasil akhir.

Langkah 10 — `loginRoutes.ts` — perlu konfirmasi Anda

File ini mendefinisikan route `/`, `/logout`, `/protected` dengan cara **manual** (`Router()` biasa, tanpa decorator) — fungsinya tumpang tindih dengan gabungan `RootController.ts` + `LoginController.ts`.

Saya tidak bisa memastikan apakah file ini masih di-mount di entry point atau merupakan sisa (leftover) dari versi sebelum proyek dipindah ke pola decorator. Karena entry point asli tidak diunggah (lihat Bagian 8), saya tidak menyertakan file ini di alur “step by step” utama. Jika Anda konfirmasi file ini masih dipakai, saya bisa update dokumen ini.

4. Entry Point (`src/index.ts`) — Rekonstruksi, Bukan File Asli

[Confidence: Low] — bagian ini **bukan** kode asli Anda. Ini adalah rekonstruksi masuk akal berdasarkan dependency di `package.json` (`express`, `cookie-session`, `reflect-metadata`) dan penggunaan `req.session` di kedua controller Anda. Silakan bandingkan dengan file asli Anda dan beri tahu saya bila berbeda supaya dokumen ini bisa diperbaiki.

```
import 'reflect-metadata';
import express from 'express';
import cookieSession from 'cookie-session';
import { AppRouter } from './AppRouter';
import './controllers/LoginController';
import './controllers/RootController';

const app = express();

app.use(express.urlencoded({ extended: true }));
app.use(
  cookieSession({
```

```

    keys: ['ganti-dengan-secret-key-anda-sendiri']
  })
});
app.use(AppRouter.getInstance());

app.listen(3000, () => {
  console.log('Listening on port 3000');
});

```

Kenapa `import './controllers/LoginController'` dan `import './controllers/RootController'` (tanpa dipakai namanya, cuma diimport)? Karena efek dari `@controller(...)` (mendaftarkan route ke `AppRouter` singleton) **terjadi saat class-nya di-load pertama kali** oleh JavaScript. Kalau file controller tidak pernah di-`import`, class-nya tidak pernah dieksekusi, decorator tidak pernah jalan, dan route tidak pernah terdaftar — walau semua kode decorator-nya benar.

5. Alur Request Secara Keseluruhan (Ringkasan)

1. Server start → entry point meng-import semua file controller → tiap `@controller(...)` langsung jalan, membaca metadata dari `@get / @post / @use / @bodyValidator`, dan mendaftar route ke `AppRouter.getInstance()` (satu Router yang sama untuk semua controller).
2. Entry point memasang `AppRouter.getInstance()` sebagai middleware Express (`app.use(...)`).
3. Saat request masuk, Express mencocokkan path & method ke route yang terdaftar, lalu menjalankan: `middlewares` (dari `@use`) → `validator` (dari `@bodyValidator`, otomatis walau kosong) → `routeHandler` (method aslinya, misalnya `postLogin`).

6. Bug yang Ditemukan pada Kode Asli & Kenapa Itu Bug

6.1 `controller.ts` — tipe `keys` salah (nyata, terverifikasi di file asli)

```

function bodyValidators(keys: string): RequestHandler {
  for (let key of keys) { /* ... */ }
}

```

`keys` dideklarasikan `string`, tapi dipakai seolah array (`for...of`, `!req.body[key]`). Karena `string` juga *iterable* di JavaScript/TypeScript (`for...of` pada `string` akan meng-iterasi karakternya satu per satu), kode ini **tidak error saat compile maupun runtime** — tapi secara logika salah total: kalau field wajib bernama `"email"`, kode akan mengecek `req.body['e']`, `req.body['m']`, `req.body['a']`, dst., bukan `req.body['email']`. Ini bug logika yang *silent* (tidak crash, tapi validasi tidak pernah bekerja seperti yang dimaksud).

[High confidence] — ini bisa dibuktikan langsung dari perilaku `for...of` terhadap `string` di JavaScript, bukan asumsi.

Perbaikan: ubah jadi `keys: string[]` (sudah diterapkan di Bagian 3 Langkah 7 di atas).

6.2 `controller.ts` — hanya cek `path`, tidak cek `method`

```

if (path) {
  router[method](...)
}

```

Kalau suatu saat ada method di class yang punya metadata `path` tapi metadata `method`-nya kosong/undefined (misalnya developer lupa memberi `@get / @post` tapi menambahkan decorator lain yang salah menyimpan `path`), `router[undefined]` akan melempar `TypeError: router[method] is not a function` saat aplikasi start — bukan pesan error yang jelas.

Perbaikan: `if (path && method)` — gagal lebih senyap (method itu cuma dilewati) daripada crash membingungkan.

6.3 `controller.ts` — tidak melewati `constructor` secara eksplisit

`Object.getOwnPropertyNames(target.prototype)` juga mengembalikan `"constructor"` sebagai salah satu nama property. Di kode asli Anda, ini **tidak menyebabkan bug nyata** karena `constructor` tidak pernah diberi metadata `path / method` oleh decorator manapun, sehingga otomatis gugur di pengecekan `if (path)`. Namun ini rapuh — sebaiknya tetap dilewati secara eksplisit sebagai pertahanan berlapis, bukan mengandalkan efek samping. Sudah ditambahkan di Bagian 3 Langkah 7.

[Medium confidence — ini rekomendasi defensif, bukan bug yang terbukti menyebabkan kegagalan di kode Anda saat ini.]

6.4 `package.json` — `typescript` dan `@types/node` tidak terdaftar

Sudah dijelaskan di Bagian 2.3. Ini bug konfigurasi paling konkret yang akan langsung menghentikan Anda di `npm run start:build`.

7. Cara Menjalankan Proyek

```
# 1) Install semua dependency (termasuk tambahan dari Bagian 2.3)
npm install

# 2) Jalankan mode development
# (menjalankan "tsc -w" dan "nodemon build/index.js" bersamaan lewat concurrently)
npm start

# Aplikasi berjalan di:
# http://localhost:3000
```

Uji manual di browser / curl:

Route	Method	Yang diharapkan
<code>/</code>	GET	Menampilkan status login (belum login pertama kali)
<code>/auth/login</code>	GET	Form login
<code>/auth/login</code>	POST (email, password)	Redirect ke <code>/</code> jika <code>hi@hi.com / password</code> , sebaliknya pesan gagal
<code>/protected</code>	GET	403 jika belum login; pesan sukses jika sudah
<code>/auth/logout</code>	GET	Menghapus session, redirect ke <code>/</code>

8. Rekomendasi Lanjutan

1. **Jalankan** `npx tsc --noEmit` secara berkala setelah menambahkan `typescript` ke devDependency — ini akan langsung menangkap bug tipe seperti `keys: string` di Bagian 6.1 sebelum jadi bug runtime.
2. **Konfirmasi status** `loginRoutes.ts` — hapus jika memang sudah tidak dipakai, supaya tidak ada dua sumber kebenaran untuk route yang sama.
3. **Tambahkan module augmentation untuk** `req.session` — assignment `req.session = { loggedIn: true }` di `LoginController.ts` kemungkinan butuh deklarasi tipe custom (`declare module 'cookie-session' { ... }` atau serupa) supaya `strict: true` tidak menandainya sebagai error. *This needs verification* — saya tidak melihat file deklarasi tipe custom di antara yang Anda unggah.
4. **Bagikan** `src/index.ts` asli kalau Anda ingin Bagian 4 di dokumen ini diganti dari rekonstruksi menjadi dokumentasi yang benar-benar akurat.